

1.1 A First Example

[This section corresponds to K&R Sec. 1.1]

The best way to learn programming is to dive right in and start writing real programs. This way, concepts which would otherwise seem abstract make sense, and the positive feedback you get from getting even a small program to work gives you a great incentive to improve it or write the next one.

Diving in with "real" programs right away has another advantage, if only pragmatic: if you're using a conventional compiler, you can't run a fragment of a program and see what it does; nothing will run until you have a complete (if tiny or trivial) program. You can't learn everything you'd need to write a complete program all at once, so you'll have to take some things "on faith" and parrot them in your first programs before you begin to understand them. (You can't learn to program just one expression or statement at a time any more than you can learn to speak a foreign language one word at a time. If all you know is a handful of words, you can't actually *say* anything: you also need to know something about the language's word order and grammar and sentence structure and declension of articles and verbs.)

Besides the occasional necessity to take things on faith, there is a more serious potential drawback of this "dive in and program" approach: it's a small step from learning-by-doing to learning-by-trial-and-error, and when you learn programming by trial-and-error, you can very easily learn many errors. When you're not sure whether something will work, or you're not even sure what you could use that might work, and you try something, and it does work, you do *not* have any guarantee that what you tried worked for the right reason. You might just have "learned" something that works only by accident or only on your compiler, and it may be very hard to un-learn it later, when it stops working.

Therefore, whenever you're not sure of something, be very careful before you go off and try it "just to see if it will work." Of course, you can never be absolutely sure that something is going to work before you try it, otherwise we'd never have to try things. But you should have an expectation that something is going to work before you try it, and if you can't predict how to do something or whether something would work and find yourself having to determine it experimentally, make a note in your mind that whatever you've just learned (based on the outcome of the experiment) is suspect.

The first example program in K&R is the first example program in any language: print or display a simple string, and exit. Here is my version of K&R's "hello, world" program:

```
#include <stdio.h>

main()
{
    printf("Hello, world!\n");
    return 0;
}
```

If you have a C compiler, the first thing to do is figure out how to type this program in and compile it and run it and see where its output went. (If you don't have a C compiler yet, the first thing to do is to find one.)

The first line is practically boilerplate; it will appear in almost all programs we write. It asks that some definitions having to do with the "Standard I/O Library" be included in our program; these definitions are needed if we are to call the library function `printf` correctly.

The second line says that we are defining a function named `main`. Most of the time, we can name our

functions anything we want, but the function name `main` is special: it is the function that will be "called" first when our program starts running. The empty pair of parentheses indicates that our `main` function accepts no *arguments*, that is, there isn't any information which needs to be passed in when the function is called.

The braces `{` and `}` surround a list of statements in C. Here, they surround the list of statements making up the function `main`.

The line

```
printf("Hello, world!\n");
```

is the first statement in the program. It asks that the function `printf` be called; `printf` is a library function which prints formatted output. The parentheses surround `printf`'s argument list: the information which is handed to it which it should act on. The semicolon at the end of the line terminates the statement.

(`printf`'s name reflects the fact that C was first developed when Teletypes and other printing terminals were still in widespread use. Today, of course, video displays are far more common. `printf`'s "prints" to the *standard output*, that is, to the default location for program output to go. Nowadays, that's almost always a video screen or a window on that screen. If you do have a printer, you'll typically have to do something extra to get a program to print to it.)

`printf`'s first (and, in this case, only) argument is the string which it should print. The string, enclosed in double quotes `"`, consists of the words "Hello, world!" followed by a special sequence: `\n`. In strings, any two-character sequence beginning with the backslash `\` represents a single special character. The sequence `\n` represents the "new line" character, which prints a carriage return or line feed or whatever it takes to end one line of output and move down to the next. (This program only prints one line of output, but it's still important to terminate it.)

The second line in the `main` function is

```
return 0;
```

In general, a function may return a value to its caller, and `main` is no exception. When `main` returns (that is, reaches its end and stops functioning), the program is at its end, and the return value from `main` tells the operating system (or whatever invoked the program that `main` is the main function of) whether it succeeded or not. By convention, a return value of 0 indicates success.

This program may look so absolutely trivial that it seems as if it's not even worth typing it in and trying to run it, but doing so may be a big (and is certainly a vital) first hurdle. On an unfamiliar computer, it can be arbitrarily difficult to figure out how to enter a text file containing program source, or how to compile and link it, or how to invoke it, or what happened after (if?) it ran. The most experienced C programmers immediately go back to this one, simple program whenever they're trying out a new system or a new way of entering or building programs or a new way of printing output from within programs. As Kernighan and Ritchie say, everything else is comparatively easy.

How *you* compile and run this (or any) program is a function of the compiler and operating system you're using. The first step is to type it in, exactly as shown; this may involve using a text editor to create a file containing the program text. You'll have to give the file a name, and all C compilers (that I've ever heard of) require that files containing C source end with the extension `.c`. So you might place the program text in a file called `hello.c`.

The second step is to compile the program. (Strictly speaking, compilation consists of two steps,

compilation proper followed by linking, but we can overlook this distinction at first, especially because the compiler often takes care of initiating the linking step automatically.) On many Unix systems, the command to compile a C program from a source file `hello.c` is

```
cc -o hello hello.c
```

You would type this command at the Unix shell prompt, and it requests that the `cc` (C compiler) program be run, placing its output (i.e. the new executable program it creates) in the file `hello`, and taking its input (i.e. the source code to be compiled) from the file `hello.c`.

The third step is to run (execute, invoke) the newly-built `hello` program. Again on a Unix system, this is done simply by typing the program's name:

```
hello
```

Depending on how your system is set up (in particular, on whether the current directory is searched for executables, based on the `PATH` variable), you may have to type

```
./hello
```

to indicate that the `hello` program is in the current directory (as opposed to some `bin` directory full of executable programs, elsewhere).

You may also have your choice of C compilers. On many Unix machines, the `cc` command is an older compiler which does not recognize modern, ANSI Standard C syntax. An old compiler will accept the simple programs we'll be starting with, but it will not accept most of our later programs. If you find yourself getting baffling compilation errors on programs which you've typed in exactly as they're shown, it probably indicates that you're using an older compiler. On many machines, another compiler called `acc` or `gcc` is available, and you'll want to use it, instead. (Both `acc` and `gcc` are typically invoked the same as `cc`; that is, the above `cc` command would instead be typed, say, `gcc -o hello hello.c`.)

(One final caveat about Unix systems: don't name your test programs `test`, because there's already a standard command called `test`, and you and the command interpreter will get badly confused if you try to replace the system's `test` command with your own, not least because your own almost certainly does something completely different.)

Under MS-DOS, the compilation procedure is quite similar. The name of the command you type will depend on your compiler (e.g. `cl` for the Microsoft C compiler, `tc` or `bcc` for Borland's Turbo C, etc.). You may have to manually perform the second, linking step, perhaps with a command named `link` or `tlink`. The executable file which the compiler/linker creates will have a name ending in `.exe` (or perhaps `.com`), but you can still invoke it by typing the base name (e.g. `hello`). See your compiler documentation for complete details; one of the manuals should contain a demonstration of how to enter, compile, and run a small program that prints some simple output, just as we're trying to describe here.

In an integrated or "visual" programming environment, such as those on the Macintosh or under various versions of Microsoft Windows, the steps you take to enter, compile, and run a program are somewhat different (and, theoretically, simpler). Typically, there is a way to open a new source window, type source code into it, give it a file name, and add it to the program (or "project") you're building. If necessary, there will be a way to specify what other source files (or "modules") make up the program. Then, there's a button or menu selection which compiles and runs the program, all from within the programming environment. (There will also be a way to create a standalone executable file which you can run from outside the environment.) In a PC-compatible environment, you may

have to choose between creating DOS programs or Windows programs. (If you have troubles pertaining to the `printf` function, try specifying a target environment of MS-DOS. Supposedly, some compilers which are targeted at Windows environments won't let you call `printf`, because until you call some fancier functions to request that a window be created, there's no window for `printf` to print to.) Again, check the introductory or tutorial manual that came with the programming package; it should walk you through the steps necessary to get your first program running.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail](#) [feedback](#)